Frontend Take-Home

~1-2 hours · AI allowed & encouraged · TypeScript + React

Gambit monitors an organization's cloud resources (buckets, databases, compute, IAM roles).
An Application is a named, logical group of resources (e.g. "Payments API"). **Build a small UI to browse the resources below, group a selection into a new Application, and view the Applications created.**
No backend - hold state in the client.

Data Model

```
type Provider = "AWS" | "GCP" | "Azure";
type Environment = "production" | "staging" | "development";
type Criticality = "low" | "medium" | "high" | "critical";

interface Resource {
  id: string; name: string; type: string;   // e.g. "S3 Bucket", "IAM Role"
  provider: Provider; region: string; environment: Environment;
  criticality: Criticality; owner: string; tags: string[];
}

interface Application { id: string; name: string; description?: string; resourceIds: string[];
}
```

Sample Resources

Here is one resource as a reference. Build out a small dataset of your own (aim for ~8-12 resources) with enough variety across providers, environments, and criticality to make search, filtering, and grouping meaningful. Be ready to explain your choices.

```
export const resources: Resource[] = [
  { id:"r-001", name:"payments-api-prod", type:"EC2 Instance", provider:"AWS",
region:"us-east-1",
    environment:"production", criticality:"critical", owner:"payments", tags:["pci","api"],
openIssues:4 },
  // ... add more resources
];
```

Core Requirements

- **Display** resources in a list/table (name, type, provider, environment, criticality, open issues).
- **Search** by name and filter by several attributes (provider / environment / criticality).
- **Select** multiple resources and create a named Application from the selection.
- **List** created Applications.
- **Visualize** an Application as a simple graph - a central Application node connected to its member resource nodes (any graph/charting library or plain SVG is fine).

Constraints

- TypeScript required; React / Remix / Next.js - your choice. Any styling approach.

Confidential - Gambit Security. For candidate evaluation only.

- Runnable from a fresh clone with one documented command (e.g. npm install && npm run dev).

Using AI & What We Evaluate

1. Don't need to adjust the UI according to our brand
2. Make sure the UI is solid and clean, it's the place to show us your eye to details
3. Use AI freely – it reflects how we work. We care about your judgement and how you shape and review its output, not whether you typed every line. We look at component structure, TypeScript quality, state modelling, UX/correctness, code clarity, and pragmatic scope choices for the time box.

Deliverables

- Git repo or zip, plus a short README: how to run it, what you built, what you'd do next, and where you used AI.

Confidential - Gambit Security. For candidate evaluation only.